Relational Database Design – Terminology Concepts Reference

Database Systems

Based on Lecture 7 by Tiantian Liu

Fall 2025

Contents

1 Notation Guide

Simple Notation Used in This Terminology Concepts Reference

Letters for Attribute Sets:

- X, Y, Z, W = Sets of attributes (columns)
- Example: X could mean {ID, name} or just {A, B}

Arrow Symbols:

- $\rightarrow$ means "determines" or "functionally determines"
- Example: $A \rightarrow B$ = "A determines B" or "If you know A, you know B"

Set Operations:

- $\subseteq$ means "is a subset of" or "is contained in"
- $\cup$ means "union" or "combine together"
- $\cap$ means "intersection" or "common elements"
- $-$ means "minus" or "remove from"

Special Notation:

- X^+ means "closure of X" = all attributes determined by X
- F^+ means "closure of F" = all FDs that can be derived
- $\{A, B\}$ = set containing A and B (often written as just AB)

2 Quick Reference Guide

Essential Concepts - At a Glance

Functional Dependencies:

- $X \rightarrow Y$ (read: "X determines Y")
- **Meaning:** If two rows have the same values in columns X, they must have the same values in columns Y
- Trivial FD: When $Y \subseteq X$ (Y is already part of X)

Armstrong's Axioms (Sound & Complete):

- Reflexivity: If $Y \subseteq X$, then $X \rightarrow Y$
- Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$
- Transitivity: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Finding Candidate Keys:

1. Enumerate all attribute combinations

2. Compute closure for each combination
3. If closure = all attributes, it's a superkey
4. Select minimal superkeys as candidate keys

Normal Forms (Strictest to Least Strict):

- BCNF $\subset$ 3NF $\subset$ 2NF $\subset$ 1NF
- Every BCNF table is also 3NF, every 3NF table is also 2NF, etc.
- Higher normal form = less redundancy = better design

3 Motivation for Good Database Design

Why Good Design Matters

Problems with Bad Design:

- **Redundancy**: Same information stored multiple times
- **Update Anomalies**: Updating one copy but not others leads to inconsistency
- **Insertion Anomalies**: Cannot insert data without other unrelated data
- **Deletion Anomalies**: Deleting data causes loss of other information

Solution: Use normalization theory based on functional dependencies to decompose relations into "good" form.

Bad Design Example from Lecture

Combining instructor and department into one table:
in_dep(i_id, name, salary, dept_name, building, budget)
Problems:

- Building and budget repeated for each instructor in the same department
- Updating Finance building requires updating multiple rows
- Risk of inconsistent updates (updating one row but not others)

Good Design:

- instructor(i_id, name, salary, dept_name)
- department(dept_name, building, budget)

4 Functional Dependencies

Functional Dependency Definition

Let R be a relation schema with $X \subseteq R$ and $Y \subseteq R$.
Plain English: Let R be a table, where X and Y are sets of columns in that table.
The functional dependency $X \to Y$ (read: "X determines Y") **holds** on R if and only if:

For any data in table R, whenever two rows have the same values in columns X, they MUST have the same values in columns Y.

Formally: $t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$
Meaning: If row 1's X values = row 2's X values, then row 1's Y values = row 2's Y values.
Trivial FD: If $Y \subseteq X$ (Y is a subset of X), then $X \to Y$ is called **trivial**.
Example: $\{A, B\} \to \{A\}$ is trivial because A is already in {A,B}.

Functional Dependency Examples

Consider relation $r(A, B, C)$:

A	B	C
a	4	5
a	4	6
b	1	2
c	8	3
d	1	5

Analysis:

- $A \to B$ holds (same A values always have same B values)
- $B \to A$ does NOT hold (e.g., B=1 appears with both A=b and A=d)

Trivial vs Non-Trivial FDs

For instructor(ID, name, salary, dept_name):
Non-Trivial FDs:

- $\{ID\} \to \{name\}$
- $\{ID\} \to \{salary, dept_name\}$

Trivial FDs:

- $\{ID, name\} \rightarrow \{name\}$ (since name $\subseteq$ {ID, name})
- $\{ID, name, salary\} \rightarrow \{salary\}$
- $\{ID, salary\} \rightarrow \{salary\}$

5 Armstrong's Axioms

Armstrong's Axioms - The Foundation

Three basic rules that are **sound** (generate only correct FDs) and **complete** (can derive all possible FDs):

1. Reflexivity Rule:

- If $Y \subseteq X$, then $X \rightarrow Y$
- **Plain English:** A set of columns always determines any subset of itself
- Example: $\{ID, name, salary\} \rightarrow \{name\}$ (the left side contains the right side)
- Example: $ABCD \rightarrow BC$ (ABCD contains BC)

2. Augmentation Rule:

- If $X \rightarrow Y$, then $XZ \rightarrow YZ$
- **Plain English:** If you add the same columns to both sides of an FD, it still holds
- Example: Given $A \rightarrow B$, we can derive $AC \rightarrow BC$
- Example: Given $\{ID\} \rightarrow \{name\}$, we can derive $\{ID, salary\} \rightarrow \{name, salary\}$

3. Transitivity Rule:

- If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
- **Plain English:** If A determines B, and B determines C, then A determines C
- Example: Given $A \rightarrow B$ and $B \rightarrow C$, we can derive $A \rightarrow C$
- Example: Given $\{ID\} \rightarrow \{dept_name\}$ and $\{dept_name\} \rightarrow \{building\}$, we can derive $\{ID\} \rightarrow \{building\}$

Proving FDs Using Armstrong's Axioms

Problem: Given $A \rightarrow B$ and $BC \rightarrow D$, prove that $AC \rightarrow D$
Proof:

1. Given: $A \rightarrow B$
2. By Augmentation: $AC \rightarrow BC$ (augment both sides with C)
3. Given: $BC \rightarrow D$
4. By Transitivity: $AC \rightarrow D$ (from steps 2 and 3)

5.1 Additional Rules (Derived from Armstrong's Axioms)

Union Rule

If $X \to Y$ holds and $X \to Z$ holds, then $X \to YZ$ holds.
Plain English: If X determines Y, and X also determines Z, then X determines both Y and Z together.
Example: Given $\{ID\} \to \{name\}$ and $\{ID\} \to \{salary\}$, we can derive $\{ID\} \to \{name, salary\}$
Proof:

1. Given: $X \to Y$, by Augmentation: $X \to XY$
2. Given: $X \to Z$, by Augmentation: $XY \to YZ$
3. By Transitivity: $X \to YZ$

Decomposition Rule

If $X \to YZ$ holds, then $X \to Y$ holds and $X \to Z$ holds.
Plain English: If X determines both Y and Z together, then X determines Y separately, and X determines Z separately.
Example: Given $\{ID\} \to \{name, salary\}$, we can derive $\{ID\} \to \{name\}$ and $\{ID\} \to \{salary\}$
Proof:

1. Since $YZ \supseteq Y$, we have $YZ \to Y$ (Reflexivity)
2. Given: $X \to YZ$ and $YZ \to Y$, we have $X \to Y$ (Transitivity)
3. Similarly for Z

Composition Rule

If $X \to Y$ holds and $Z \to W$ holds, then $XZ \to YW$ holds.
Plain English: If X determines Y, and Z determines W, then X combined with Z determines Y combined with W.
Example: Given $A \to B$ and $C \to D$, we can derive $AC \to BD$
Proof:

1. Given: $X \to Y$, by Augmentation: $XZ \to YZ$
2. Given: $Z \to W$, by Augmentation: $YZ \to YW$
3. By Transitivity: $XZ \to YW$

Pseudotransitivity Rule

If $X \to Y$ holds and $YZ \to W$ holds, then $XZ \to W$ holds.
Proof:

1. Given: $X \to Y$, by Augmentation: $XZ \to YZ$
2. Given: $YZ \to W$ and $XZ \to YZ$, by Transitivity: $XZ \to W$

6 Closure of Attribute Sets

Attribute Closure Definition

Given a set of attributes X, the **closure of X under F** (denoted by X^+) is the set of attributes that are functionally determined by X under F.
Plain English: X^+ is the set of ALL columns that can be determined by knowing X.
Think of it as: If you know the values in columns X, what other columns can you figure out? That's X^+.

Algorithm to Compute Attribute Closure

To compute X^+ (the closure of X under F):

```
result := X;                            // Start with X
while (changes to result) do            // Keep going until nothing changes
    for each Y -> Z in F do             // Check each FD
        begin
            if Y is subset of result    // If Y is in result
            then result := result + Z   // Add Z to result
        end
```

Plain English Algorithm:

1. Start with result = X (the columns you're starting with)
2. Look at each functional dependency $Y \to Z$ in F:
   - If Y is already in result, add Z to result
   - **Why?** Because if we have Y, we can figure out Z
3. Repeat step 2 until no more columns can be added
4. Return result as X^+

Computing Attribute Closure - Example 1

Given:

- $R = (A, B, C, G, H, I)$
- $F = \{A \to B, A \to C, CG \to H, CG \to I, B \to H\}$

Compute $(AG)^+$:

1. result = $\{A, G\}$ (start with AG)
2. A $\subseteq$ {A, G}, so add B: result = $\{A, B, G\}$ (from $A \to B$)
3. A $\subseteq$ {A, B, G}, so add C: result = $\{A, B, C, G\}$ (from $A \to C$)
4. CG $\subseteq$ {A, B, C, G}, so add H: result = $\{A, B, C, G, H\}$ (from $CG \to H$)
5. CG $\subseteq$ {A, B, C, G, H}, so add I: result = $\{A, B, C, G, H, I\}$ (from $CG \to I$)
6. No more changes

Answer: $(AG)^+ = \{A, B, C, G, H, I\}$

Computing Attribute Closure - Example 2

Given:

- $R = (A, B, C, D, E, F)$
- $F = \{AB \to C, AD \to E, B \to D, AF \to B\}$

Compute $(AB)^+$:

1. result = $\{A, B\}$
2. AB $\subseteq$ {A, B}, so add C: result = $\{A, B, C\}$ (from $AB \to C$)
3. B $\subseteq$ {A, B, C}, so add D: result = $\{A, B, C, D\}$ (from $B \to D$)
4. AD $\subseteq$ {A, B, C, D}, so add E: result = $\{A, B, C, D, E\}$ (from $AD \to E$)
5. No more changes

Answer: $(AB)^+ = \{A, B, C, D, E\}$

Using Closure to Verify FDs

To prove that $X \to Y$ holds:
Simply show that $(X)^+$ contains Y.
Example: Given $F = \{AB \to C, AD \to E, B \to D, AF \to B\}$, prove that $AF \to D$
Solution: Compute $(AF)^+$:

1. result = $\{A, F\}$
2. AF $\subseteq$ {A, F}, so add B: result = $\{A, B, F\}$ (from $AF \to B$)
3. AB $\subseteq$ {A, B, F}, so add C: result = $\{A, B, C, F\}$ (from $AB \to C$)
4. B $\subseteq$ {A, B, C, F}, so add D: result = $\{A, B, C, D, F\}$ (from $B \to D$)
5. AD $\subseteq$ {A, B, C, D, F}, so add E: result = $\{A, B, C, D, E, F\}$ (from $AD \to E$)

$(AF)^+ = \{A, B, C, D, E, F\}$ which contains D, therefore $AF \to D$ holds.

7 Keys and Superkeys

Superkey and Candidate Key

Superkey:

- K is a superkey for relation schema R if and only if $K \to R$ (K determines all of R)
- **Plain English:** A set of columns that uniquely identifies every row
- To check if K is a superkey: Compute K^+. If $K^+ = R$ (all columns), then K is a superkey.

Candidate Key:

- K is a candidate key for R if and only if:

1. $K \to R$ (K is a superkey)
2. K is minimal (no smaller subset of K is also a superkey)

- **Plain English:** If we remove ANY column from a candidate key, it stops being a superkey

Primary Key:

- One of the candidate keys chosen by the database designer

Superkey vs Candidate Key

For instructor(ID, name, salary, dept_name):
Candidate Key:

- $\{ID\}$ (minimal set that determines all attributes)

Superkeys (not candidate keys):

- $\{ID, name\}$
- $\{ID, salary\}$
- $\{ID, name, salary\}$
- $\{ID, name, salary, dept_name\}$

All superkeys contain the candidate key $\{ID\}$ but have additional attributes.

7.1 Finding Candidate Keys - Complete Algorithm

Algorithm to Find All Candidate Keys

Step 1: Enumerate all possible attribute combinations

- Start with single attributes: A, B, C, ...
- Then pairs: AB, AC, BC, ...
- Then triples: ABC, ABD, ...
- Continue until all attributes: ABC...Z

Step 2: For each combination, compute its closure
Step 3: If a closure contains all attributes, it's a superkey
Step 4: Select only minimal superkeys as candidate keys

- If both A and AB are superkeys, only A is a candidate key

Finding Candidate Keys - Complete Example

Given:

- $R = (A, B, C, D)$
- $F = \{AB \to C, AD \to B, B \to D\}$

Step 1: Enumerate combinations:

- Size 1: A, B, C, D
- Size 2: AB, AC, AD, BC, BD, CD
- Size 3: ABC, ABD, ACD, BCD
- Size 4: ABCD

Step 2 & 3: Compute closures and identify superkeys:

- $(A)^+ = A$ (not a superkey)
- $(B)^+ = BD$ (not a superkey)
- $(C)^+ = C$ (not a superkey)
- $(D)^+ = D$ (not a superkey)
- $(AB)^+ = ABCD$ ✓(superkey!)
- $(AC)^+ = AC$ (not a superkey)
- $(AD)^+ = ABCD$ ✓(superkey!)
- $(BC)^+ = BCD$ (not a superkey)
- $(BD)^+ = BD$ (not a superkey)
- $(CD)^+ = CD$ (not a superkey)

Step 4: Select candidate keys:

- AB is a superkey and minimal (neither A nor B alone is a superkey)
- AD is a superkey and minimal (neither A nor D alone is a superkey)
- Skip ABC, ABD, ACD, ABCD (not minimal since AB and AD are already superkeys)

Answer: Candidate keys are AB and AD

7.2 Tricks for Finding Candidate Keys Faster

Trick 1: Check Small Combinations First

Always start with the smallest combinations. If you find candidate keys among single attributes or pairs, you don't need to check larger combinations.
Example:

- $R = (A, B, C, D)$
- $F = \{A \to B, B \to C, C \to D, D \to A\}$

Compute closures:

- $(A)^+ = ABCD$ ✓

- $(B)^+ = ABCD$ ✓
- $(C)^+ = ABCD$ ✓
- $(D)^+ = ABCD$ ✓

No need to check others! All larger combinations are superkeys but not candidate keys.
Candidate keys: A, B, C, D

Trick 2: Attributes Not on Right Side Must Be in Every Key

If an attribute does not appear on the right side of any FD, it must be in every candidate key.
Example:

- $R = (A, B, C, D)$
- $F = \{AB \to C, AD \to B, B \to D\}$

Observation: A does not appear on the right side of any FD.
Conclusion: A must be in every candidate key.
Strategy: Only check combinations that include A:

- $(A)^+ = A$ (not enough)
- $(AB)^+ = ABCD$ ✓
- $(AC)^+ = AC$ (not enough)
- $(AD)^+ = ABCD$ ✓

Candidate keys: AB and AD

8 Decomposition

Decomposition Definition

Let R be a relation schema and let R_1 and R_2 form a decomposition of R.
That is: $R = R_1 \cup R_2$
Plain English: Split table R into two smaller tables R_1 and R_2 such that all columns of R are covered.
Goal: Eliminate redundancy by splitting relations.
Requirements for a Good Decomposition:

1. **Lossless Decomposition**: When you join the two tables back together, you get exactly the original data
2. **Dependency Preserving**: You can verify all the FDs by looking at each small table separately

8.1 Lossless Decomposition

Lossless Decomposition

A decomposition of R into R_1 and R_2 is **lossless** if:
When you take table r, split it into R_1 and R_2, then join them back together, you get exactly the original table r.
No extra rows, no missing rows.
Test for Lossless Decomposition:
R_1 and R_2 is a lossless decomposition if we can derive at least one of:

- $(R_1 \cap R_2) \rightarrow R_1$ – common attributes are superkey in R_1, OR
- $(R_1 \cap R_2) \rightarrow R_2$ – common attributes are superkey in R_2

Simple Test: Find the common columns $(R_1 \cap R_2)$. Check if they're a superkey in at least one of the tables.

Lossless Decomposition Example

Given:

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Decomposition: $R_1 = (A, B)$, $R_2 = (B, C)$

Check if lossless:

- $R_1 \cap R_2 = \{B\}$
- Is B a superkey in R_1? Check: $(B)^+ = BC$. Does it contain all of $R_1 = AB$? No.
- Is B a superkey in R_2? Check: $(B)^+ = BC$. Does it contain all of $R_2 = BC$? Yes!

Answer: Lossless decomposition (since $B \rightarrow BC$ holds)

Lossy Decomposition Example

Given:

- instructor(ID, name, salary, dept_name)
- Decomposition:
  - R_1 = (ID, name)
  - R_2 = (name, salary, dept_name)

Problem: Two instructors with the same name (e.g., "Califieri")
When we join back, we get spurious tuples (extra wrong rows).
Check:

- $R_1 \cap R_2 = \{name\}$
- Is name a superkey? No! (Multiple instructors can have the same name)
- Therefore: Lossy decomposition

8.2 Dependency Preserving Decomposition

Dependency Preserving

A decomposition of R into $R_1, R_2, ..., R_n$ is **dependency preserving** if:
All functional dependencies that hold for R can be verified locally in the decomposed schemas $R_1, ..., R_n$.
Benefit: We don't need to compute expensive joins to verify constraints.

Dependency Preserving Example

Given:

- $R = (A, B, C)$
- $F = \{A \to B, B \to C\}$
- Decomposition: $R_1 = (A, B)$, $R_2 = (B, C)$

Check dependency preserving:

- $A \to B$ can be checked locally in R_1 ✓
- $B \to C$ can be checked locally in R_2 ✓

Answer: Dependency preserving!

9 Normal Forms

Normal Forms Overview

Normal Forms define characteristics of relational schemas to avoid redundancies and anomalies.
Hierarchy (Strictest to Least Strict):

$$\text{BCNF} \subset \text{3NF} \subset \text{2NF} \subset \text{1NF}$$

Higher normal forms have less redundancy but may sacrifice dependency preservation.

9.1 First Normal Form (1NF)

First Normal Form (1NF)

A relation R is in **1NF** if the domains of all its attributes are **atomic**.
Atomic means:

- No composite attributes
- No set-valued attributes
- No multivalued attributes

In other words: Each cell contains only a single value.

Converting to 1NF

Not in 1NF:

i_id	name	course_id
10101	Srinivasan	CS-101, CS-120, CS-131
12121	Wu	FIN-101
15151	Mozart	MUS-101, MUS-145

Problem: course_id is set-valued (multiple courses in one cell)
In 1NF:

i_id	name	course_id
10101	Srinivasan	CS-101
10101	Srinivasan	CS-120
10101	Srinivasan	CS-131
12121	Wu	FIN-101
15151	Mozart	MUS-101
15151	Mozart	MUS-145

Each course is in a separate row.

9.2 Second Normal Form (2NF)

Second Normal Form (2NF)

A relation R is in **2NF** if:

1. It is in 1NF, AND
2. Every non-key attribute is functionally dependent on the **whole** of every candidate key

Violation: If a non-key attribute depends on only part of a candidate key (partial dependency).

2NF Example

Relation:

i_id	name	course_id
10101	Srinivasan	CS-101
10101	Srinivasan	CS-120
12121	Wu	FIN-101

Analysis:

- Candidate key: $\{i_id, course_id\}$
- FD: $i_id \rightarrow name$ (name depends on only part of the key!)
- This is a **partial dependency**

Answer: NOT in 2NF
Solution: Decompose into:

- instructor(i_id, name)
- teaches(i_id, course_id)

9.3 Third Normal Form (3NF)

Third Normal Form (3NF)

A relation schema R is in **3NF** if for all $X \rightarrow Y$ in F^+, at least one of the following holds:

1. $X \rightarrow Y$ is trivial (i.e., $Y \subseteq X$)
2. X is a superkey for R
3. Each attribute A in $Y - X$ is contained in a candidate key for R

Simple Check: For each FD, either:

- (1) it's trivial, OR
- (2) left side is a superkey, OR
- (3) right side attributes are in candidate keys

Checking 3NF

Given:

- instructor(ID, name, dept_name, building)
- Candidate key: $\{ID\}$
- FDs:
  - $\{ID\} \rightarrow \{name, dept_name, building\}$
  - $\{dept_name\} \rightarrow \{building\}$

Check 3NF:
For FD $\{dept_name\} \rightarrow \{building\}$:

1. Is it trivial? No (building is not in dept_name)
2. Is dept_name a superkey? No
3. Is building contained in a candidate key? No

Answer: NOT in 3NF (violates condition 3)

9.4 Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF)

A relation schema R is in **BCNF** if for all $X \to Y$ in F^+, at least one of the following holds:

1. $X \to Y$ is trivial (i.e., $Y \subseteq X$)
2. X is a superkey for R

Difference from 3NF: No third option! Every non-trivial FD must have a superkey on the left side.
Plain English Rule: For every functional dependency (except trivial ones), the left side MUST be a superkey.
BCNF is stricter than 3NF:

- Every BCNF relation is in 3NF
- Not every 3NF relation is in BCNF
- BCNF eliminates more redundancy

Checking BCNF

Given:

- instructor(ID, name, dept_name, building)
- Candidate key: $\{ID\}$
- FDs:
  - $\{ID\} \to \{name, dept_name, building\}$
  - $\{dept_name\} \to \{building\}$

Check BCNF:
For FD $\{dept_name\} \to \{building\}$:

1. Is it trivial? No
2. Is dept_name a superkey? No

Answer: NOT in BCNF
Solution - Decompose:

- R_1 = instructor(ID, name, dept_name)
- R_2 = department(dept_name, building)

Now both relations are in BCNF.

Quick Comparison: 3NF vs BCNF

3NF allows: Non-trivial FDs where left side is not a superkey, IF the right side attributes are part of some candidate key.

BCNF requires: For all non-trivial FDs, the left side MUST be a superkey.
Trade-off:

- BCNF: No redundancy, but may not preserve dependencies
- 3NF: Allows some redundancy, but always preserves dependencies

10 Step-by-Step Guide to Database Design Problems

10.1 How to Check if a Relation is in a Normal Form

Procedure to Check Normal Forms

Check 1NF:

1. Look at each attribute
2. Verify each attribute has atomic values (no sets, no lists)

Check 2NF:

1. Verify relation is in 1NF
2. Find all candidate keys
3. Check if any non-key attribute depends on only part of a candidate key
4. If yes: NOT in 2NF

Check 3NF:

1. Verify relation is in 2NF
2. For each FD $X \to Y$ in F:
   - Check if $X \to Y$ is trivial, OR
   - Check if X is a superkey, OR
   - Check if each attribute in $Y - X$ is in some candidate key
3. If all FDs satisfy at least one condition: in 3NF

Check BCNF:

1. For each FD $X \to Y$ in F:
   - Check if $X \to Y$ is trivial, OR
   - Check if X is a superkey
2. If all FDs satisfy at least one condition: in BCNF

10.2 How to Decompose into BCNF

BCNF Decomposition Algorithm

Input: Relation R with FDs F
Output: A lossless decomposition of R into BCNF
Algorithm:

1. result := {R} (start with the original table)
2. done := false
3. While (not done) do:
   - If there exists R_i in result that is not in BCNF, then:
     (a) Find an FD $X \to Y$ in R_i that violates BCNF (X is NOT a superkey)
     (b) Replace R_i with:
       - $R_{i1} = X \cup Y$ (left side + right side)
       - $R_{i2} = R_i - (Y - X)$ (everything except attributes only on right)
   - Else done := true

Simple Steps:

1. Find a violating FD (left side is not a superkey)
2. Split the table using that FD
3. Repeat until all tables are in BCNF

Note: Result is always lossless but may not preserve dependencies.

BCNF Decomposition Example

Given:

- $R =$ instructor(ID, name, dept_name, building)
- $F = \{ID \to name, ID \to dept_name, ID \to building, dept_name \to building\}$
- Candidate key: $\{ID\}$

Step 1: Check if in BCNF

- FD $dept_name \to building$ violates BCNF (dept_name is not a superkey)

Step 2: Decompose using the violating FD

- X = {dept_name}, Y = {building}
- $R_1 = X \cup Y =$ department(dept_name, building)
- $R_2 = R - (Y - X) =$ instructor(ID, name, dept_name)

Step 3: Check if both are in BCNF

- R_1: Only FD is $dept_name \to building$, and dept_name is a candidate key. BCNF! ✓

- R_2: FDs are $ID \to name, ID \to dept_name$, and ID is a candidate key. BCNF! ✓

Final Answer:

- department(dept_name, building)
- instructor(ID, name, dept_name)

10.3 How to Decompose into 3NF

3NF Decomposition Algorithm

Input: Relation R with FDs F
Output: A lossless and dependency-preserving decomposition of R into 3NF
Algorithm:

1. Compute a canonical cover F_c of F
2. For each FD $X \to Y$ in F_c:
   - Create a relation schema $R_i = XY$
3. If none of the schemas contains a candidate key of R:
   - Add a schema containing a candidate key
4. Remove redundant schemas (if one is contained in another)

Note: Result is always lossless and dependency-preserving.

11 Summary - Key Points for Exam

Essential Formulas and Rules

Armstrong's Axioms:

- Reflexivity: $Y \subseteq X \Rightarrow X \to Y$
- Augmentation: $X \to Y \Rightarrow XZ \to YZ$
- Transitivity: $(X \to Y \text{ and } Y \to Z) \Rightarrow X \to Z$

Additional Rules:

- Union: $(X \to Y \text{ and } X \to Z) \Rightarrow X \to YZ$
- Decomposition: $X \to YZ \Rightarrow X \to Y \text{ and } X \to Z$
- Composition: $(X \to Y \text{ and } Z \to W) \Rightarrow XZ \to YW$
- Pseudotransitivity: $(X \to Y \text{ and } YZ \to W) \Rightarrow XZ \to W$

Keys:

- Superkey: $K^+ = R$
- Candidate key: Minimal superkey

Lossless Decomposition Test:

- $(R_1 \cap R_2) \to R_1$ OR $(R_1 \cap R_2) \to R_2$

Normal Forms:

- 1NF: Atomic values
- 2NF: 1NF + No partial dependencies
- 3NF: For all $X \to Y$: trivial OR X is superkey OR $Y - X$ in candidate key
- BCNF: For all $X \to Y$: trivial OR X is superkey

Common Exam Question Types

Type 1: Compute Attribute Closure

- Use the closure algorithm
- Start with given attributes
- Iteratively apply FDs until no more attributes can be added

Type 2: Find Candidate Keys

- Look for attributes not on right side (must be in key)
- Enumerate combinations starting from smallest
- Compute closures
- Select minimal superkeys

Type 3: Check Normal Form

- Find all candidate keys first
- Check each FD against the normal form criteria
- Identify violating FDs

Type 4: Check Lossless Decomposition

- Find $R_1 \cap R_2$ (common attributes)
- Check if common attributes are superkey in R_1 or R_2

Type 5: Decompose into BCNF/3NF

- Find violating FD
- Split relation based on violating FD
- Repeat until all relations satisfy the normal form

12 Practice Exercises

12.1 Exercise 1: Identifying Functional Dependencies

Practice - FD Identification

Given the following relation instance:

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d2
a2	b2	c3	d1
a3	b1	c4	d3
a3	b1	c4	d3

Questions:

1. Does $A \to B$ hold?
2. Does $B \to A$ hold?
3. Does $AB \to C$ hold?
4. Does $AC \to D$ hold?

Tips for solving:

- For $X \to Y$ to hold: whenever two rows have same X values, they must have same Y values
- One counterexample disproves an FD

Solutions:

1. $A \to B$: **YES** - Rows 1,2 have A=a1 and B=b1; Rows 4,5 have A=a3 and B=b1
2. $B \to A$: **NO** - Rows 1 and 4 have B=b1 but different A (a1 vs a3)
3. $AB \to C$: **NO** - Rows 1,2 have AB=(a1,b1) but different C (c1 vs c2)
4. $AC \to D$: **YES** - All AC combinations are unique

12.2 Exercise 2: Computing Attribute Closure

Practice - Attribute Closure Computation

Given:

- $R = (A, B, C, D, E, F, G)$
- $F = \{A \to BC, CD \to E, E \to F, AC \to G, B \to D\}$

Questions:

1. Compute $(A)^+$
2. Compute $(AC)^+$
3. Compute $(CE)^+$

Solutions:
1. Compute $(A)^+$:

- result $= \{A\}$
- $A \to BC$: add B, C $\to$ result $= \{A, B, C\}$
- $B \to D$: add D $\to$ result $= \{A, B, C, D\}$
- $CD \to E$: add E $\to$ result $= \{A, B, C, D, E\}$
- $E \to F$: add F $\to$ result $= \{A, B, C, D, E, F\}$
- $AC \to G$: add G $\to$ result $= \{A, B, C, D, E, F, G\}$
- **Answer:** $(A)^+ = \{A, B, C, D, E, F, G\}$ (all attributes!)

2. Compute $(AC)^+$:

- Same process: $(AC)^+ = \{A, B, C, D, E, F, G\}$

3. Compute $(CE)^+$:

- result $= \{C, E\}$
- $E \to F$: add F $\to$ result $= \{C, E, F\}$
- No more FDs apply
- **Answer:** $(CE)^+ = \{C, E, F\}$

12.3 Exercise 3: Finding Candidate Keys

Practice - Candidate Key Discovery

Problem:

- $R = (A, B, C, D, E)$
- $F = \{A \to B, BC \to D, D \to E, E \to A\}$

Find all candidate keys for R.

Hint: Which attribute(s) never appear on the right side of any FD?
Solution:
Observation: C never appears on right side → C must be in every key
Compute closures with C:

- $(C)^+ = \{C\}$ (not enough)
- $(AC)^+$: A → B → {A,B,C} → BC → D → {A,B,C,D} → D → E → {A,B,C,D,E} ✓
- $(BC)^+$: BC → D → {B,C,D} → D → E → {B,C,D,E} → E → A → {A,B,C,D,E} ✓
- $(CD)^+$: D → E → {C,D,E} → E → A → {A,C,D,E} → A → B → {A,B,C,D,E} ✓
- $(CE)^+$: E → A → {A,C,E} → A → B → {A,B,C,E} → BC → D → {A,B,C,D,E} ✓

Answer: Candidate keys are {AC, BC, CD, CE}

12.4 Exercise 4: Proving Functional Dependencies

Practice - FD Proofs Using Armstrong's Axioms

Problem 1: Given $A \to C$ and $B \to D$, prove that $AB \to CD$
Proof:

1. Given: $A \to C$ (Augmentation with B) $\Rightarrow AB \to CB$
2. Given: $B \to D$ (Augmentation with A) $\Rightarrow AB \to AD$
3. From steps 1 and 2: $AB \to CB$ and $AB \to AD$ (Union) $\Rightarrow AB \to ABCD$
4. By Decomposition: $AB \to CD$ ✓

Problem 2: Given $A \to B$, $B \to C$, $C \to D$, prove $A \to D$
Proof:

1. Given: $A \to B$ and $B \to C$ (Transitivity) $\Rightarrow A \to C$
2. From step 1: $A \to C$ and Given: $C \to D$ (Transitivity) $\Rightarrow A \to D$ ✓

Problem 3: Given $AB \to C$ and $C \to D$, prove $AB \to D$
Proof:

1. Given: $AB \to C$ and $C \to D$ (Transitivity) $\Rightarrow AB \to D$ ✓

Problem 4 (Pseudo-Transitivity II): Given $X \to YZ$ and $Y \to W$, prove $X \to YWZ$
Proof:

1. Given: $X \to YZ$
2. Given: $Y \to W$
3. From (1), by Decomposition: $X \to Y$
4. From (3) and (2), by Transitivity: $X \to W$

5. From (1) and (4), by Union: $X \to YZW$ ✓

Conclusion: Since the order of attributes on the right side doesn't matter, $X \to YZW$ is equivalent to $X \to YWZ$.
Key axioms used:

- Decomposition: $X \to YZ \Rightarrow X \to Y$
- Transitivity: $X \to Y, Y \to Z \Rightarrow X \to Z$
- Union: $X \to Y, X \to Z \Rightarrow X \to YZ$

12.5 Exercise 5: Checking Lossless Decomposition

Practice - Lossless Decomposition Check

Problem:

- $R = (A, B, C, D)$
- $F = \{A \to B, B \to C, C \to D\}$
- Decomposition: $R_1 = (A, B, C)$, $R_2 = (C, D)$

Is this decomposition lossless?
Solution:

- $R_1 \cap R_2 = \{C\}$
- Is C a superkey in R_1? Compute $(C)^+ = \{C, D\}$. Does NOT contain A, B. No.
- Is C a superkey in R_2? $(C)^+ = \{C, D\}$. Contains C, D. Yes!

Answer: YES, lossless (because $C \to CD$ holds)

12.6 Exercise 6: Normal Form Identification

Practice - Checking Normal Forms

Problem:

- $R =$ student_course(student_id, student_name, course_id, grade)
- Candidate key: $\{student_id, course_id\}$
- FDs: $student_id \to student_name$

Questions:

1. Is R in 1NF?
2. Is R in 2NF?
3. Is R in 3NF?
4. Is R in BCNF?

Solutions:
1. Is in 1NF? YES - all attributes are atomic
2. Is in 2NF? NO

- Candidate key: $\{student_id, course_id\}$
- FD: $student_id \rightarrow student_name$
- student_name depends on only part of key (student_id)
- This is a **partial dependency** → violates 2NF

3. Is in 3NF? NO (not even in 2NF)
4. Is in BCNF? NO (not even in 2NF)

12.7 Exercise 7: BCNF Decomposition

Practice - Decomposing into BCNF

Problem:

- $R =$ booking(customer_id, hotel_id, room_no, date, price, hotel_city)
- Candidate key: $\{hotel_id, room_no, date\}$
- FDs:
  - $hotel_id, room_no, date \rightarrow customer_id, price, hotel_city$
  - $hotel_id \rightarrow hotel_city$
  - $hotel_id, date \rightarrow price$

Solution:
Step 1: Check if in BCNF

- $hotel_id \rightarrow hotel_city$ - Left side NOT superkey. VIOLATES BCNF
- $hotel_id, date \rightarrow price$ - Left side NOT superkey. VIOLATES BCNF

Step 2: Decompose using $hotel_id \rightarrow hotel_city$

- $R_1 =$ hotel(hotel_id, hotel_city)
- $R_2 =$ booking2(customer_id, hotel_id, room_no, date, price)

Step 3: Check R_2 for BCNF

- FD in R_2: $hotel_id, date \rightarrow price$ - Left side NOT superkey
- Decompose R_2:
  - $R_3 =$ room_pricing(hotel_id, date, price)
  - $R_4 =$ booking_final(customer_id, hotel_id, room_no, date)

Final BCNF Decomposition:

1. hotel(hotel_id, hotel_city)
2. room_pricing(hotel_id, date, price)
3. booking_final(customer_id, hotel_id, room_no, date)

All three relations are now in BCNF ✓

12.8 Quick Self-Check Questions

Rapid Review Questions

Answer True or False:

1. If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ (T/F)
2. Every BCNF relation is in 3NF (T/F)
3. Every 3NF relation is in BCNF (T/F)
4. If X is a superkey, then $X^+ = R$ (T/F)
5. A relation can have multiple candidate keys (T/F)
6. Trivial FDs are always valid (T/F)

7. Lossless decomposition means no data is lost (T/F)
8. BCNF eliminates all redundancy (T/F)
9. If an attribute doesn't appear on right side of any FD, it must be in every key (T/F)

Answers: 1-T, 2-T, 3-F, 4-T, 5-T, 6-T, 7-T, 8-T (mostly), 9-T